

BAB 4

INHERITANCE (PEWARISAN)

4.1 Pendahuluan

Pada BAB 3, mahasiswa telah mempelajari konsep **enkapsulasi**, yaitu bagaimana melindungi data dalam class menggunakan access modifier serta mengaksesnya melalui getter dan setter. Dengan konsep tersebut, program menjadi lebih aman dan terstruktur.

Pada BAB 4 ini, mahasiswa akan mempelajari konsep **inheritance (pewarisan)**. Konsep ini memungkinkan sebuah class untuk mewarisi atribut dan method dari class lain. Dengan inheritance, kita tidak perlu menulis ulang kode yang sama, sehingga program menjadi lebih efisien dan mudah dikembangkan.

Inheritance merupakan salah satu pilar utama dalam pemrograman berorientasi objek. Pemahaman yang baik pada bab ini akan sangat membantu mahasiswa dalam memahami konsep lanjutan seperti polymorphism pada bab berikutnya.

4.2 Tujuan Pembelajaran

Setelah menyelesaikan praktikum pada bab ini, mahasiswa diharapkan mampu memahami dan mengimplementasikan konsep inheritance dalam Java.

Secara lebih rinci, mahasiswa diharapkan mampu:

1. Menjelaskan konsep inheritance dalam OOP
2. Memahami hubungan superclass dan subclass
3. Menggunakan keyword `extends`
4. Mengakses atribut dan method dari class induk
5. Menggunakan keyword `super`
6. Menerapkan inheritance dalam studi kasus sederhana
7. Memahami manfaat penggunaan inheritance

4.3 Pengertian Inheritance

Inheritance atau pewarisan adalah konsep di mana sebuah class dapat **mewarisi atribut dan method dari class lain**.

Class yang diwarisi disebut:

- **Superclass (parent class / class induk)**

Class yang mewarisi disebut:

- **Subclass (child class / class turunan)**

Dengan inheritance, subclass dapat menggunakan kembali kode dari superclass tanpa harus menulis ulang. Hal ini membuat program menjadi lebih ringkas dan terstruktur.

4.4 Analogi Sederhana Inheritance

Agar konsep inheritance lebih mudah dipahami, perhatikan contoh pada dunia hewan. Misalnya, kita memiliki sebuah class **Hewan** yang berisi data dan perilaku umum yang dimiliki banyak hewan.

Semua hewan pada contoh ini memiliki atribut umum seperti:

- nama
- umur

Semua hewan juga memiliki method umum seperti:

- makan()
- tidur()

Namun, beberapa hewan memiliki ciri atau perilaku tambahan yang berbeda.

Kucing memiliki tambahan:

- ras
- mengeong()
- berburu()

Anjing memiliki tambahan:

- jenisGolongan
- menggonggong()
- bermain()

Dalam hal ini:

- **Hewan** → superclass (kelas induk)
- **Kucing** dan **Anjing** → subclass (kelas anak)

Artinya, class **Kucing** dan **Anjing** mewarisi atribut dan method umum dari class **Hewan**, lalu menambahkan atribut dan method yang menjadi ciri khusus masing-masing.

Inheritance juga bisa terjadi secara bertingkat. Misalnya, dari class **Anjing** dapat dibuat lagi class **GoldenRetriever**.

GoldenRetriever memiliki tambahan:

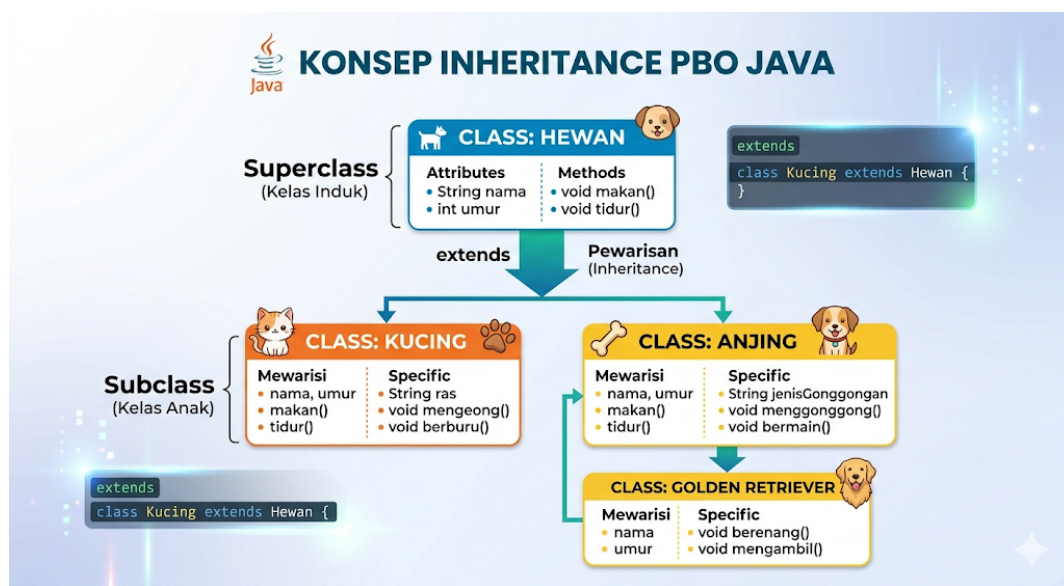
- `berenang()`
- `mengambil()`

Dalam hal ini:

- **Anjing** → superclass bagi `GoldenRetriever`
- **GoldenRetriever** → subclass dari `Anjing`

Karena `GoldenRetriever` adalah turunan dari `Anjing`, maka class ini mewarisi semua yang dimiliki `Anjing`. Selain itu, karena `Anjing` juga merupakan turunan dari `Hewan`, maka `GoldenRetriever` secara tidak langsung juga mewarisi atribut dan method dari `Hewan`.

Untuk mempermudah memahami hubungan antara class induk dan class turunan, perhatikan ilustrasi berikut.



Gambar 4.1 Konsep Inheritance (Pewarisan) dalam Pemrograman Java

Gambar ini menunjukkan bahwa class **Hewan** menjadi class induk yang memiliki atribut dan method umum. Class **Kucing** dan **Anjing** menjadi class turunan yang mewarisi bagian umum tersebut, lalu menambahkan bagian khusus masing-masing. Sementara itu, class **GoldenRetriever** merupakan turunan dari **Anjing**, sehingga contoh ini juga menunjukkan **pewarisan bertingkat** (*multilevel inheritance*).

Dari ilustrasi tersebut dapat dilihat bahwa kita tidak perlu menulis ulang atribut dan method yang sama pada setiap class. Cukup definisikan bagian yang umum pada class induk, lalu tambahkan bagian yang khusus pada class turunannya.

4.5 Syntax Inheritance dalam Java

Setelah memahami konsep inheritance melalui analogi pada subbab sebelumnya, sekarang kita akan melihat bagaimana konsep tersebut ditulis dalam bentuk program Java.

Dalam Java, inheritance digunakan dengan keyword: **extends**

Keyword `extends` digunakan untuk menyatakan bahwa sebuah class merupakan turunan dari class lain.

4.5.1 Bentuk Umum Inheritance

Bentuk dasar penulisan inheritance dalam Java adalah sebagai berikut:

```
class NamaSubclass extends NamaSuperclass {  
  
    // atribut dan method tambahan  
  
}
```

Artinya, `NamaSubclass` akan mewarisi semua atribut dan method dari `NamaSuperclass`.

4.5.1 Contoh Sederhana

Perhatikan contoh berikut:

```
class Hewan {  
  
    String nama;  
  
}
```

Kemudian kita membuat class turunan:

```
class Kucing extends Hewan {  
  
    String ras;  
  
}
```

Pada contoh ini, class `Kucing` secara otomatis memiliki atribut `nama` dari class `Hewan`, walaupun atribut tersebut tidak ditulis ulang di dalam class `Kucing` serta menambahkan atribut baru yaitu `ras`.

4.6 Contoh Program Dasar Inheritance

Setelah memahami analogi inheritance melalui class **Hewan**, **Kucing**, dan **Anjing**, sekarang kita masuk ke bentuk programnya di Java. Pada contoh pertama ini, kita mulai dari bentuk yang paling sederhana terlebih dahulu agar mahasiswa memahami hubungan antara class induk dan class turunan.

Class **Hewan** akan menjadi superclass. Class ini berisi atribut umum seperti `nama` dan `umur`, serta method umum seperti `makan()` dan `tidur()`.

Superclass Hewan

```
class Hewan {  
    String nama;  
    int umur;  
  
    void makan() {  
        System.out.println(nama + " sedang makan.");  
    }  
  
    void tidur() {  
        System.out.println(nama + " sedang tidur.");  
    }  
}
```

Pada class `Hewan`, kita mendefinisikan hal-hal yang umum dimiliki banyak hewan. Jadi, jika nanti ada class `Kucing` atau `Anjing`, kita tidak perlu menulis ulang atribut `nama`, `umur`, method `makan()`, dan `tidur()`.

subclass Kucing

```
class Kucing extends Hewan {
    String ras;

    void mengeong() {
        System.out.println(nama + " sedang mengeong.");
    }

    void berburu() {
        System.out.println(nama + " sedang berburu tikus.");
    }
}
```

Pada contoh ini, class `Kucing` mewarisi semua yang ada di class `Hewan`. Jadi, object `Kucing` otomatis memiliki atribut `nama` dan `umur`, serta method `makan()` dan `tidur()`, walaupun keduanya tidak ditulis ulang di dalam class `Kucing`.

Program Utama

```
public class DemoInheritanceHewan {
    public static void main(String[] args) {
        Kucing k1 = new Kucing();

        k1.nama = "Milo";
        k1.umur = 2;
        k1.ras = "Persia";

        System.out.println("Nama Kucing : " + k1.nama);
        System.out.println("Umur      : " + k1.umur + " tahun");
        System.out.println("Ras      : " + k1.ras);

        k1.makan();
        k1.tidur();
        k1.mengeong();
        k1.berburu();
    }
}
```

Output:

```
Nama Kucing : Milo
Umur      : 2 tahun
Ras      : Persia
Milo sedang makan.
Milo sedang tidur.
Milo sedang mengeong.
Milo sedang berburu tikus.
```

Dari contoh ini, terlihat bahwa object `k1` dari class `Kucing` dapat menggunakan method milik class `Hewan`. Inilah inti inheritance, yaitu subclass dapat langsung menggunakan atribut dan method yang diwarisi dari superclass.

4.7 Memahami Alur Pewarisan

Pada tahap ini, mahasiswa perlu benar-benar memahami alurnya. Saat kita menulis `class Kucing extends Hewan`, artinya `class Kucing` adalah turunan dari `class Hewan`.

Akibatnya, semua anggota dari `class Hewan` yang dapat diwariskan akan menjadi milik `class Kucing`. Jadi, `object Kucing` tidak hanya memiliki ciri khas kucing, tetapi juga membawa semua sifat umum yang sudah didefinisikan pada `Hewan`.

Alurnya dapat dipahami seperti ini:

- `Hewan` menyimpan hal-hal umum
- `Kucing` mewarisi hal-hal umum dari `Hewan`
- `Kucing` menambahkan hal-hal khusus miliknya sendiri

Dengan cara ini, program menjadi lebih rapi. Kita tidak perlu menulis `method makan()` dan `tidur()` berulang kali untuk `Kucing`, `Anjing`, atau `class hewan` lain.

4.8 Menambahkan Subclass Lain: Anjing

Agar konsep inheritance semakin jelas, sekarang kita tambahkan subclass lain, yaitu `Anjing`. `Class` ini juga mewarisi atribut dan `method` dari `Hewan`, tetapi memiliki atribut dan `method` khusus yang berbeda dengan `Kucing`.

4.8.1 Class Anjing

```
class Anjing extends Hewan {
    String jenisGolongan;

    void menggonggong() {
        System.out.println(nama + " sedang menggonggong.");
    }

    void bermain() {
        System.out.println(nama + " sedang bermain bola.");
    }
}
```

4.8.2 Program Utama

```
public class DemoInheritanceAnjing {
    public static void main(String[] args) {
        Anjing a1 = new Anjing();

        a1.nama = "Buddy";
        a1.umur = 3;
        a1.jenisGolongan = "Anjing Penjaga";

        System.out.println("Nama Anjing      : " + a1.nama);
        System.out.println("Umur           : " + a1.umur + " tahun");
        System.out.println("Jenis Golongan  : " + a1.jenisGolongan);

        a1.makan();
        a1.tidur();
        a1.menggonggong();
        a1.bermain();
    }
}
```

Output:

```
Nama Anjing      : Buddy
Umur           : 3 tahun
Jenis Golongan  : Anjing Penjaga
Buddy sedang makan.
Buddy sedang tidur.
Buddy sedang menggonggong.
Buddy sedang bermain bola.
```

Dari contoh ini, mahasiswa bisa membandingkan dua subclass berbeda yang berasal dari superclass yang sama. Walaupun `Kucing` dan `Anjing` sama-sama berasal dari `Hewan`, keduanya tetap bisa memiliki karakteristik yang berbeda.

4.9 Keyword Super

Setelah memahami bahwa subclass mewarisi anggota dari superclass, langkah berikutnya adalah mengenal keyword `super`. Keyword `super` digunakan untuk mengakses anggota milik superclass dari dalam subclass.

Keyword ini sangat berguna, terutama ketika kita ingin memanggil constructor superclass atau mengakses method superclass secara eksplisit. Pada tahap awal, mahasiswa cukup memahami dua penggunaan penting, yaitu untuk constructor dan method.

4.9.1 Super untuk Memanggil Constructor Superclass

Perhatikan contoh berikut.

```
class Hewan {
    String nama;
    int umur;

    Hewan(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
    }
}
```

Pada class `Hewan`, kita membuat constructor yang langsung mengisi `nama` dan `umur`. Sekarang, jika kita membuat class `Kucing`, kita bisa menggunakan `super()` untuk memanggil constructor tersebut.

```
class Kucing extends Hewan {
    String ras;

    Kucing(String nama, int umur, String ras) {
        super(nama, umur);
        this.ras = ras;
    }

    void tampilData() {
        System.out.println("Nama : " + nama);
        System.out.println("Umur : " + umur + " tahun");
        System.out.println("Ras : " + ras);
    }
}
```

```
public class DemoSuperConstructor {
    public static void main(String[] args) {
        Kucing k1 = new Kucing("Milo", 2, "Persia");
        k1.tampilData();
    }
}
```

4.9.1 Program Utama

Output:

Nama : Milo
Umur : 2 tahun

Ras : Persia

Pada contoh ini, `super(nama, umur);` berarti class `Kucing` memanggil constructor milik class `Hewan`. Jadi, pengisian atribut umum tidak perlu ditulis ulang.

4.10 Super untuk Memanggil Method Superclass

Selain constructor, `super` juga bisa digunakan untuk memanggil method dari superclass.

```
class Hewan {  
  
    void info() {  
  
        System.out.println("Ini adalah hewan.");  
  
    }  
  
}
```

```
class Anjing extends Hewan {  
  
    void info() {  
  
        super.info();  
  
        System.out.println("Anjing adalah turunan dari class Hewan.");  
  
    }  
  
}
```

```
public class DemoSuperMethod {  
  
    public static void main(String[] args) {  
  
        Anjing a1 = new Anjing();  
  
        a1.info();  
  
    }  
  
}
```

Output:

Ini adalah hewan.

Anjing adalah turunan dari class Hewan.

4.11 Inheritance Bertingkat (*Multilevel Inheritance*)

Pada ilustrasi sebelumnya, class **GoldenRetriever** merupakan turunan dari class **Anjing**. Ini adalah contoh **inheritance bertingkat**, yaitu ketika sebuah subclass diturunkan lagi menjadi subclass baru.

Alurnya menjadi seperti ini:

- Hewan → class induk paling umum
- Anjing → turunan dari Hewan
- GoldenRetriever → turunan dari Anjing

Artinya, `GoldenRetriever` mewarisi:

- atribut dan method dari `Anjing`
- atribut dan method dari `Hewan` secara tidak langsung

4.12 Contoh Program

```
class Hewan {  
    String nama;  
    int umur;  
  
    void makan() {  
        System.out.println(nama + " sedang makan.");  
    }  
}
```

```
class Anjing extends Hewan {  
    void menggonggong() {  
        System.out.println(nama + " sedang menggonggong.");  
    }  
}
```

```
class GoldenRetriever extends Anjing {  
    void berenang() {  
        System.out.println(nama + " suka berenang.");  
    }  
  
    void mengambil() {  
        System.out.println(nama + " sedang mengambil bola.");  
    }  
}
```

```
public class DemoMultilevelInheritance {  
    public static void main(String[] args) {  
        GoldenRetriever g1 = new GoldenRetriever();  
  
        g1.nama = "Max";  
        g1.umur = 4;  
  
        System.out.println("Nama : " + g1.nama);  
        System.out.println("Umur : " + g1.umur + " tahun");  
  
        g1.makan();  
        g1.menggonggong();  
        g1.berenang();  
        g1.mengambil();  
    }  
}
```

Output:

Nama : Max

Umur : 4 tahun

Max sedang makan.

Max sedang menggonggong.

Max suka berenang.

Max sedang mengambil bola.

Contoh ini sangat penting karena menunjukkan bahwa inheritance bisa membentuk hierarki yang bertingkat. Mahasiswa perlu memahami bahwa semakin ke bawah, class biasanya menjadi semakin spesifik.

4.13 Keuntungan Inheritance

Inheritance memiliki beberapa keuntungan yang sangat penting dalam pemrograman berorientasi objek.

1. Mengurangi Duplikasi Kode

Jika semua hewan punya method `makan()` dan `tidur()`, maka cukup tulis sekali di class `Hewan`. Kita tidak perlu menyalin method yang sama ke class `Kucing`, `Anjing`, dan `GoldenRetriever`.

2. Membuat Program Lebih Rapi

Dengan inheritance, bagian yang umum diletakkan di superclass, sedangkan bagian khusus diletakkan di subclass. Akibatnya, struktur class menjadi lebih jelas dan lebih mudah dipahami.

3. Memudahkan Pengembangan Program

Jika suatu saat kita ingin menambahkan method `bernapas()` untuk semua hewan, kita cukup menambahkannya di class `Hewan`. Semua subclass akan otomatis mendapat method tersebut.

4. Mendukung Reusability

Kode yang sudah ditulis pada superclass dapat digunakan kembali oleh subclass. Inilah salah satu kekuatan utama OOP, yaitu memanfaatkan kembali kode yang sudah ada.

4.14 Hal-Hal yang Perlu Diperhatikan

Walaupun inheritance sangat berguna, penggunaannya tetap harus tepat. Tidak semua hubungan antar class harus dibuat menggunakan inheritance.

Inheritance cocok dipakai jika hubungan antar class benar-benar menunjukkan hubungan “**adalah**” atau “**is-a**”. Misalnya:

- Kucing adalah hewan
- Anjing adalah hewan
- GoldenRetriever adalah anjing

Sebaliknya, kalau hubungannya hanya “memiliki” atau “has-a”, maka biasanya lebih cocok menggunakan relasi lain seperti agregasi atau komposisi, yang akan dibahas di bab lain.

4.15 Kesalahan Umum pada Inheritance

Pada saat belajar inheritance, ada beberapa kesalahan yang sering dilakukan mahasiswa.

1. Salah Memahami Hubungan Class

Mahasiswa kadang membuat inheritance hanya karena ingin “menghubungkan” dua class. Padahal inheritance hanya tepat jika memang ada hubungan “adalah”.

2. Menulis Ulang Atribut dan Method yang Sudah Diwarisi

Jika `Kucing` sudah mewarisi `nama` dan `umur` dari `Hewan`, maka atribut itu tidak perlu ditulis ulang di class `Kucing`, kecuali memang ada alasan khusus.

3. Lupa Menggunakan `extends`

Tanpa keyword `extends`, Java tidak akan menganggap class tersebut sebagai subclass.

4. Tidak Memahami Fungsi `super`

Mahasiswa kadang memakai `super` hanya menyalin contoh tanpa paham kegunaannya. Padahal `super` dipakai untuk mengakses constructor atau method dari superclass.

5. Mencampur Contoh yang Tidak Konsisten

Saat belajar inheritance, akan lebih mudah jika contoh dibuat dalam satu tema yang sama. Karena itu, pada BAB 4 ini semua contoh dibuat tetap dalam konteks hewan agar alurnya tidak membingungkan.

4.16 Tugas Percobaan

Percobaan 1

Buatlah class `Hewan` yang memiliki atribut:

- `nama`
- `umur`

Tambahkan method:

- `makan()`
- `tidur()`

Percobaan 2

Buatlah class `Kucing` yang merupakan turunan dari `Hewan`. Tambahkan:

- atribut `ras`
- method `mengeong()`
- method `berburu()`

Lalu buat satu object `Kucing` dan tampilkan semua data serta method yang dimilikinya.

Percobaan 3

Buatlah class `Anjing` yang merupakan turunan dari `Hewan`. Tambahkan:

- atribut `jenisGolongan`
- method `menggonggong()`
- method `bermain()`

Lalu buat satu object `Anjing` dan tampilkan semua data serta method yang dimilikinya.

Percobaan 4

Ubah class `Hewan`, `Kucing`, dan `Anjing` agar menggunakan constructor. Gunakan keyword `super` pada subclass.

Percobaan 5

Buatlah class `GoldenRetriever` sebagai turunan dari `Anjing`. Tambahkan:

- method `berenang()`
- method `mengambil()`

Lalu buat satu object `GoldenRetriever` dan tunjukkan bahwa object tersebut dapat menggunakan:

- method dari `Hewan`
- method dari `Anjing`
- method miliknya sendiri

4.17 Pertanyaan Diskusi

1. Apa yang dimaksud dengan inheritance dalam OOP?
2. Apa perbedaan superclass dan subclass?
3. Mengapa inheritance dapat mengurangi duplikasi kode?
4. Apa fungsi keyword `extends`?
5. Apa fungsi keyword `super`?
6. Mengapa `GoldenRetriever` dapat menggunakan method milik `Hewan`?
7. Apa yang dimaksud dengan multilevel inheritance?
8. Kapan inheritance sebaiknya digunakan?

4.18 Tugas Laporan Praktikum

Setelah menyelesaikan seluruh percobaan, mahasiswa diminta membuat laporan praktikum. Format laporan tetap dibuat konsisten dengan BAB sebelumnya agar mahasiswa terbiasa dengan pola kerja yang rapi.

Struktur laporan:

1. Cover
2. Tujuan praktikum
3. Alat dan bahan
4. Langkah percobaan
5. Source code
6. Hasil output program
7. Analisis program
8. Kesimpulan

Ketentuan isi analisis:

Pada bagian analisis, mahasiswa harus menjelaskan:

- class mana yang menjadi superclass
- class mana yang menjadi subclass
- atribut dan method apa saja yang diwariskan
- penggunaan `extends`
- penggunaan `super`
- perbedaan antara inheritance biasa dan inheritance bertingkat

Laporan dikumpulkan dalam bentuk **PDF** sesuai ketentuan dosen atau asisten praktikum.

4.19 Penutup

Pada BAB 4 ini, mahasiswa telah mempelajari inheritance sebagai salah satu konsep paling fundamental dalam pemrograman berorientasi objek. Melalui inheritance, mahasiswa belajar bahwa sebuah class dapat mewarisi atribut dan method dari class lain, sehingga program menjadi lebih ringkas, rapi, dan mudah dikembangkan.

Jika konsep ini benar-benar dipahami, maka mahasiswa akan lebih siap mempelajari **polymorphism** pada bab berikutnya. Sebab, polymorphism sangat erat kaitannya dengan inheritance dan biasanya jauh lebih mudah dipahami jika pondasi pewarisan class sudah kuat.